NRQL Reference Guide — Commands, Functions and Extraction Queries

Oct 28, 2026 · @Vivek Arora

1. About NRQL and this guide

NRQL (New Relic Query Language) is the SQL-like language for querying all telemetry stored in New Relic's database (NRDB). Every chart in New Relic — APM summaries, dashboards, alert conditions — is an NRQL query underneath. If you can read and write NRQL, you can answer questions the built-in screens do not, build your own dashboards and alerts, and pull raw data out for reports.

NRQL is **read-only and time-bound**: every query covers a time window (one hour by default) and returns aggregated values, raw rows or time series. It looks like SQL, but there are no tables to create, no `UPDATE` or `INSERT`, and joins are limited to subqueries.

Where you run NRQL

Place	How to open it	Typical use
Query builder	*Query your data* in the left navigation (or the `/` search → *Query your data*)	Ad-hoc questions, testing a query, exporting CSV/JSON
Dashboards	Any widget → *Edit* (each widget is one query)	Charts and tables people look at every day
Alert conditions	*Alerts* → *Alert conditions* → *New alert condition* → *Write your own query*	Continuous evaluation of one numeric value
NerdGraph API	`https://api.newrelic.com/graphql` (EU: `api.eu.newrelic.com/graphql`) with a user key	Scripts, reports, data extraction
New Relic CLI	`newrelic nrql query --accountId <id> --query "…"`	Shell scripts and quick checks
Notebooks	*Notebooks* → query block	Investigations written up as you go

What NRQL can read

Data type	What it is	Examples of what you query
Events	One record per thing that happened, with attributes	`Transaction`, `TransactionError`, `PageView`, `PageAction`, `SyntheticCheck`, `SystemSample`, `ProcessSample`, `K8sPodSample`, `NrAiIncident`, `NrAuditEvent`, custom events
Metrics	Pre-aggregated numeric data points, dimensional or timeslice	`FROM Metric SELECT …` (Prometheus, OpenTelemetry, cloud integrations, APM timeslice metrics such as `apm.service.transaction.duration`)
Logs	Log lines with attributes	`FROM Log SELECT …`
Traces	Spans of distributed traces	`FROM Span SELECT …`

To see which data types your account has, run `SHOW EVENT TYPES SINCE 1 week ago`. To see the attributes of one type, run `SELECT keyset() FROM Transaction SINCE 1 day ago`.

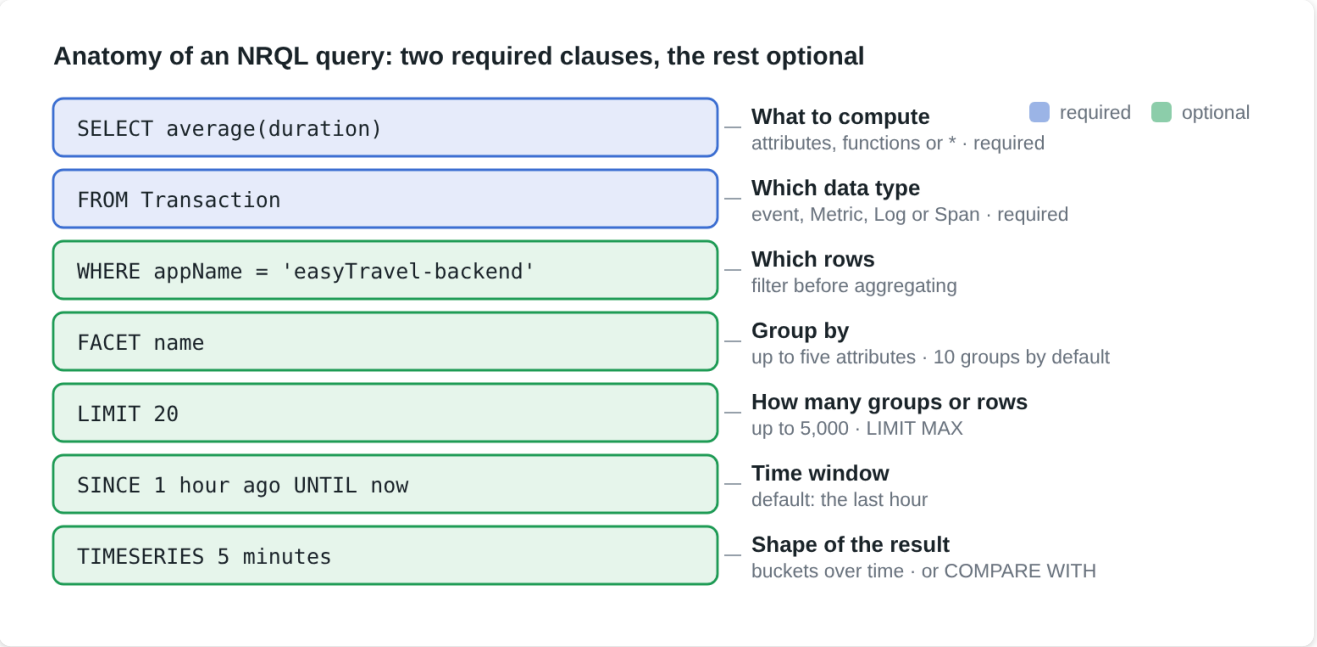
How to use this guide

- **Sections 2–5** are the language: anatomy, every clause, every major function, and advanced techniques.

- **Section 6** is a cookbook of extraction queries by data source — copy, change the names, run.
- **Section 7** covers getting data *out* (CSV, JSON, NerdGraph, CLI).
- **Sections 8–10** cover alerts, limits and a one-page cheat sheet.
- Examples use the easyTravel application from the Day 1 labs (`easyTravel-frontend`, `easyTravel-backend`). Replace application names, hosts and attribute names with your own.
- Pictures labelled **Expected view** are redrawn New Relic screens for training: the layout and numbers on your account will differ, but the *shape* of the result — a table, a time series, a comparison — is what to look for.

> **Conventions.** Keywords are written in UPPER CASE for readability; NRQL keywords are case-insensitive, but **attribute names and event type names are case-sensitive** (`appName` is not `appname`).

2. Query anatomy and syntax rules



The clauses of an NRQL query, in the order they are usually written

The basic template

```
SELECT function(attribute) [AS 'label'], ...
FROM DataType [, DataType ...]
[WHERE condition [AND|OR condition ...]]
[FACET attribute [, attribute ...] | FACET CASES (...)]
[LIMIT number | LIMIT MAX] [OFFSET number]
[ORDER BY attribute [ASC|DESC]]
[SINCE time] [UNTIL time]
[TIMESERIES interval | AUTO | MAX] [SLIDE BY interval]
[COMPARE WITH time ago]
[WITH TIMEZONE 'Area/City']
```

Rules that save time

Rule	Detail
Two clauses are required	`SELECT` and `FROM`. Either one may come first: `FROM Transaction SELECT count(*)` is the same query.
The rest is optional	Optional clauses can be written in any order after `SELECT` / `FROM`; the order above is the convention.
Default time window	One hour (`SINCE 1 hour ago`) when no `SINCE` is given.
Case	Keywords are case-insensitive. Event types, attribute names and string values in `=` comparisons are case-sensitive. `LIKE` is case-insensitive.
Strings	Use single quotes: `WHERE appName = 'easyTravel-backend'`.
Odd attribute names	Backticks around names with spaces, dots that are not nesting, or reserved words: `SELECT average(`response.time`)`.

Rule	Detail
Labels	`AS 'Average (s)'` renames a column; labels with spaces need quotes.
Comments	`--` or `//` to the end of the line, `/* … */` for blocks.
Several data types	`FROM Transaction, TransactionError` queries both; use `eventType()` to tell rows apart.
Several accounts	Queries run in one account; cross-account queries are done through NerdGraph or dashboards (up to 30 accounts).

WHERE operators

Operator	Example
`=` `!=` `<` `<=` `>` `>=`	`WHERE duration > 0.5`
`AND` `OR` `NOT` and parentheses	`WHERE (httpResponseCode = '500' OR error IS true) AND appName = 'easyTravel-backend'`
`IN` / `NOT IN`	`WHERE request.method IN ('POST', 'PUT')`
`LIKE` / `NOT LIKE` (`%` = any characters, case-insensitive)	`WHERE name LIKE '%Booking%'`
`RLIKE` / `NOT RLIKE` (RE2 regular expression, must match the whole value)	`WHERE request.uri RLIKE '/services/[A-Za-z]+Service.*'`
`IS NULL` / `IS NOT NULL`	`WHERE error.class IS NOT NULL`
`IS TRUE` / `IS FALSE`	`WHERE error IS true`

Arithmetic in SELECT

`+ - * /` work on attributes and on aggregated results:

```
-- Arithmetic on attributes, then aggregate
SELECT average(duration - databaseDuration) AS 'Non-DB time (s)'
FROM Transaction WHERE appName = 'easyTravel-backend' SINCE 1 day ago

-- Arithmetic on aggregated results
SELECT count(*) / uniqueCount(session) AS 'Pages per session'
FROM PageView SINCE 1 day ago
```

3. Clause reference

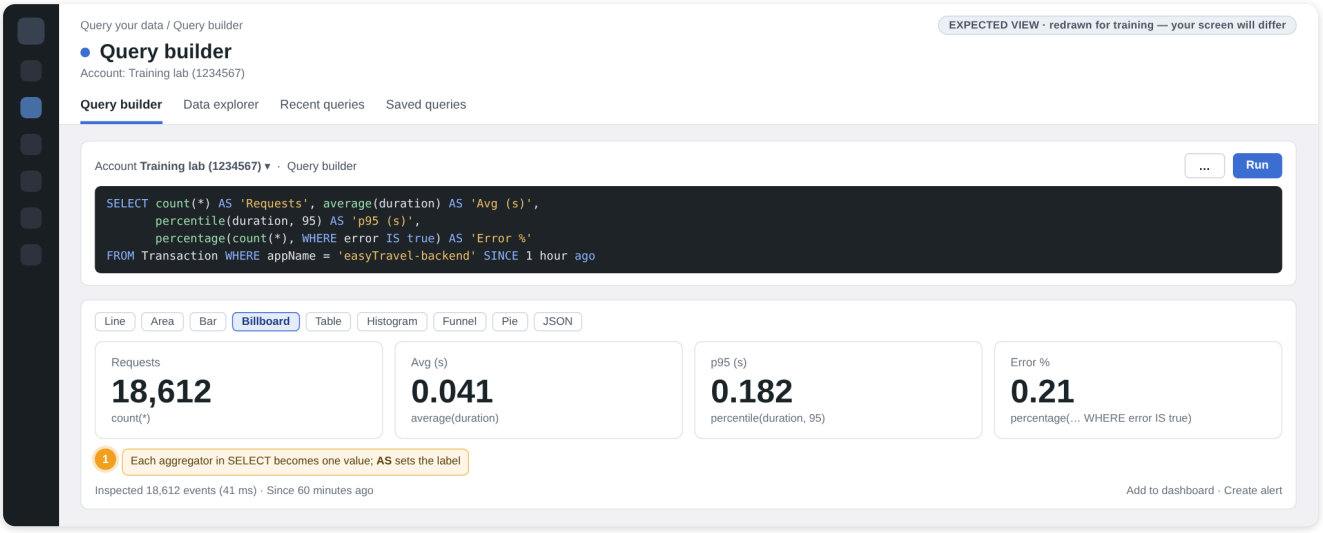
Each clause below has its syntax, what it does, the details that trip people up, and an example you can run against the easyTravel data.

SELECT — what to return

```
SELECT attribute | function(attribute) | * | expression [AS 'label'], ...
```

- `SELECT *` returns raw rows (all attributes, 100 rows by default, up to 5,000 with `LIMIT MAX`).
- `SELECT attr1, attr2` returns raw rows with only those columns — the basis of data extraction.
- `SELECT function(attr)` returns aggregated values. You can mix several aggregators in one query, but not aggregators **and** plain attributes (use `FACET` or `latest()` instead).
- `AS 'label'` names a result column.

```
SELECT count(*) AS 'Requests', average(duration) AS 'Avg (s)',
       percentile(duration, 95) AS 'p95 (s)',
       percentage(count(*), WHERE error IS true) AS 'Error %'
FROM Transaction WHERE appName = 'easyTravel-backend' SINCE 1 hour ago
```



Expected view — four aggregators, four labelled values (billboard)

FROM — which data

```
FROM EventType [, EventType ...]
FROM Metric | Log | Span
```

- One or more data types, comma-separated. `eventType()` returns the type of each row when you query several.
- `FROM Metric` queries dimensional metrics; APM timeslice metrics are exposed as names such as `apm.service.transaction.duration`.
- `SHOW EVENT TYPES [SINCE …]` replaces `SELECT … FROM` to list the types in the account.

WHERE — which rows

```
WHERE condition [AND|OR condition ...]
```

Filters **before** aggregation. All operators are in section 2. Two tips: put the most selective condition first for readability (NRDB optimizes either way), and remember `=` on strings is exact and case-sensitive while `LIKE` is case-insensitive.

```
SELECT count(*) FROM TransactionError
WHERE appName = 'easyTravel-backend' AND error.class NOT LIKE '%Timeout%'
SINCE 6 hours ago
```

FACET — group by

```
FACET attribute [AS 'label'] [, attribute ...]   -- up to 5 attributes
FACET attribute ORDER BY function(attribute)      -- choose which facets win
FACET CASES (WHERE cond [AS 'label'], ...) [OR attribute]
```

- Returns one row (or one line, with `TIMESERIES`) per distinct value. **Default 10 facets**, up to 5,000 with `LIMIT`.
- Facets are ranked by the **first** aggregator in `SELECT`, highest first. `FACET … ORDER BY` ranks by another aggregator (always descending).
- In tables, values beyond the limit are simply not returned; faceted time series charts group them into an *Other* line. Raise `LIMIT` when totals matter.
- Rows that do not have the attribute fall into an empty facet.

```
SELECT average(duration) FROM Transaction
WHERE appName = 'easyTravel-frontend'
FACET request.uri SINCE 1 hour ago LIMIT 5
```



Expected view — FACET request.uri: one bar per page, top five by average duration

LIMIT and OFFSET — how many

```
LIMIT number | LIMIT MAX [OFFSET number]
```

- With `FACET`: number of groups (default 10). With `SELECT *` or attribute lists: number of rows (default 100).
- Maximum 5,000; `LIMIT MAX` always means the current maximum.
- `OFFSET n` skips the first *n* rows (rows are newest first) — useful to page through raw rows within the limit.

ORDER BY — sort raw rows

```
ORDER BY attribute [ASC|DESC]
```

Sorts raw-row results (default `DESC` for timestamps). It does not apply to faceted queries — use `FACET … ORDER BY` there.

```
SELECT timestamp, name, duration FROM Transaction
WHERE appName = 'easyTravel-backend' ORDER BY duration DESC LIMIT 20
```

SINCE and UNTIL — the time window

```
SINCE time  [UNTIL time]
```

Form	Examples
Relative	`SINCE 30 minutes ago`, `SINCE 2 hours ago UNTIL 1 hour ago`, `SINCE 1 week ago`
Calendar	`SINCE today`, `SINCE yesterday UNTIL today`, `SINCE this week`, `SINCE last month UNTIL this month`
Absolute	`SINCE '2026-10-27 09:00:00' UNTIL '2026-10-27 17:00:00'` (account time zone unless `WITH TIMEZONE`)
Epoch	`SINCE 1793023320000` (milliseconds since 1970)

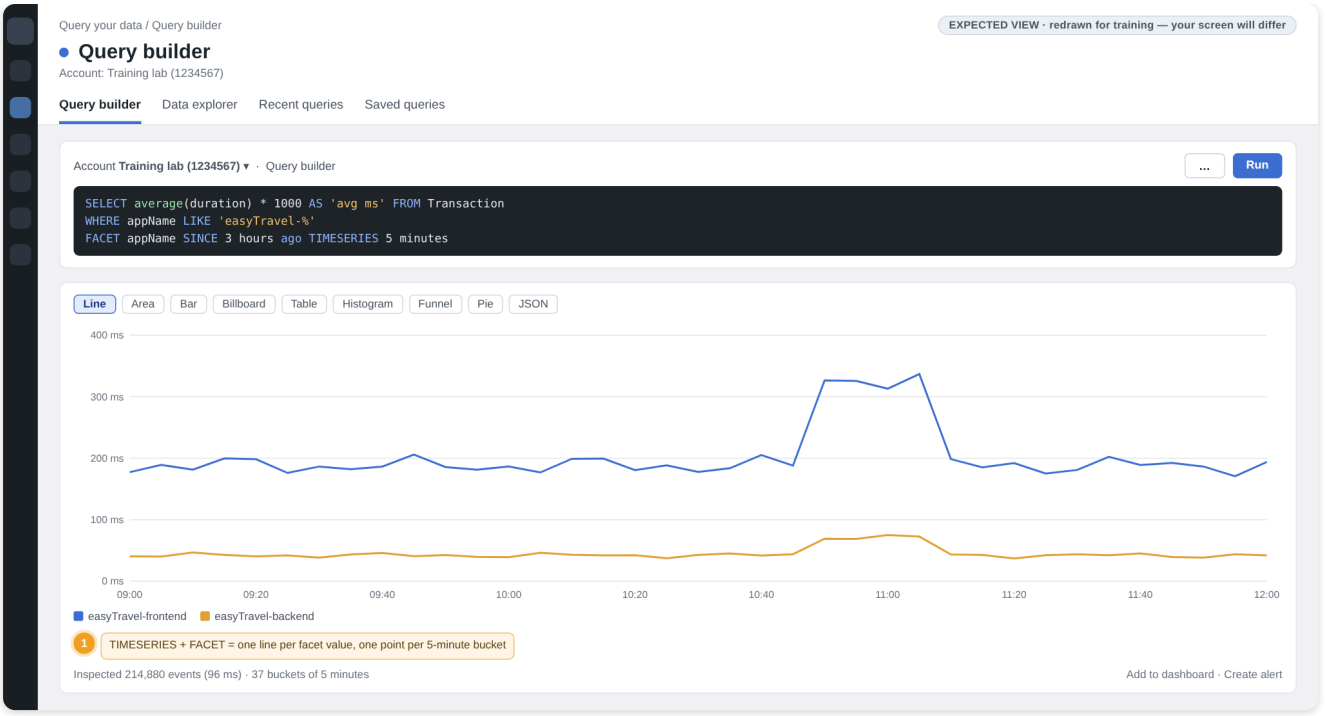
Defaults: `SINCE 1 hour ago`, `UNTIL now`. `UNTIL` is exclusive. Long windows read more data and run slower — narrow the window before adding detail.

TIMESERIES — results over time

```
TIMESERIES [interval | AUTO | MAX] [SLIDE BY interval | AUTO]
```

- Splits the window into buckets and returns one value per bucket (a line chart).
- `TIMESERIES` alone or `AUTO` picks a sensible bucket size; `MAX` gives the smallest allowed bucket (at most 366 buckets).
- `SLIDE BY` makes overlapping windows, which smooths spiky data: `TIMESERIES 5 minutes SLIDE BY 1 minute`.
- Combine with `FACET` for one line per value.

```
SELECT average(duration) * 1000 AS 'avg ms' FROM Transaction
WHERE appName LIKE 'easyTravel-%'
FACET appName SINCE 3 hours ago TIMESERIES 5 minutes
```



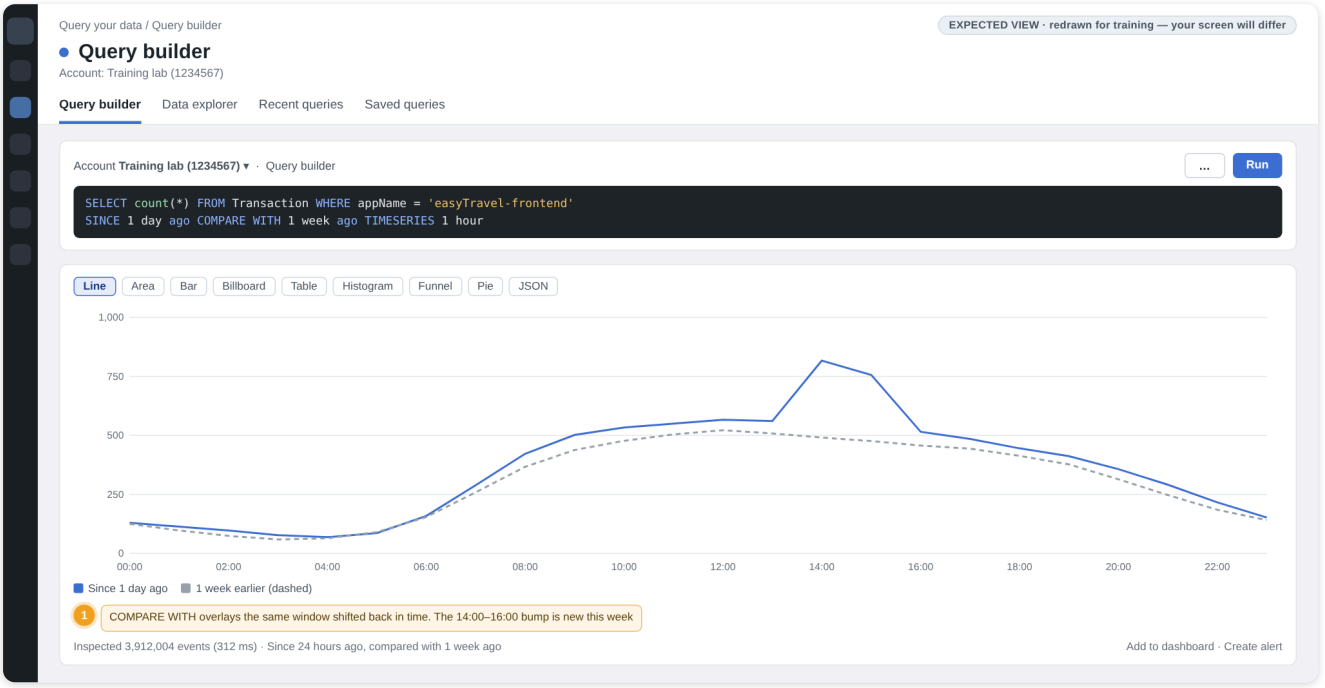
Expected view — TIMESERIES with FACET: one line per application, 5-minute buckets

COMPARE WITH — now versus before

```
COMPARE WITH time ago
```

Runs the same query for a window shifted back by the given time and returns both. It needs a `SINCE` (or `UNTIL`). Works with and without `TIMESERIES`; without it you get the two values and the % change.

```
SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-frontend'
SINCE 1 day ago COMPARE WITH 1 week ago TIMESERIES 1 hour
```



Expected view — COMPARE WITH: today's traffic over the same day last week (dashed)

WITH TIMEZONE — calendar boundaries

```
WITH TIMEZONE 'Asia/Kolkata'
```

Changes how `SINCE today`, absolute dates and time functions (`hourOf()`, `weekdayOf()`) are interpreted. Use IANA names (`Europe/London`, `America/New_York`).

```
SELECT count(*) FROM PageView FACET hourOf(timestamp)
SINCE today WITH TIMEZONE 'Asia/Kolkata' LIMIT 24
```

EXTRAPOLATE — estimate sampled totals

APM agents send a limited number of `Transaction` events per minute. When traffic is very high, events are sampled; `EXTRAPOLATE` scales `count`, `sum`, `rate`, `average`, `percentage`, `histogram`, `apdex` and `stddev` back up. It applies to `Transaction`, `TransactionError` and custom APM events.

```
SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-backend' SINCE 1 day ago
EXTRAPOLATE
```

PREDICT — a forecast line

Adds a forecast to a `TIMESERIES` line chart. It cannot be combined with `COMPARE WITH`.

```
SELECT count(*) FROM Transaction SINCE 1 week ago TIMESERIES 1 hour PREDICT
```

WITH … AS — variables

```
WITH row_function(...) AS variable [, ...]
WITH aparse(attribute, 'pattern') AS (var1, var2, ...)
```

Names the result of a row-wise function once and reuses it in `SELECT`, `WHERE` and `FACET`. Only one `WITH` per query; aggregators cannot be variables.

```
FROM Transaction WITH round(duration * 1000) AS ms
SELECT average(ms), max(ms) WHERE ms > 500 FACET name SINCE 1 hour ago
```

JOIN and subqueries

Covered with examples in section 5: `FROM A JOIN (subquery) ON key SELECT …`, scalar subqueries in `WHERE`, `IN (subquery)` and nested aggregation `FROM (SELECT …)`.

> **Not a query: DELETE.** `DELETE FROM … WHERE …` exists only in *Pipeline Control* drop rules, to stop data being stored. You cannot delete stored data from the query builder.

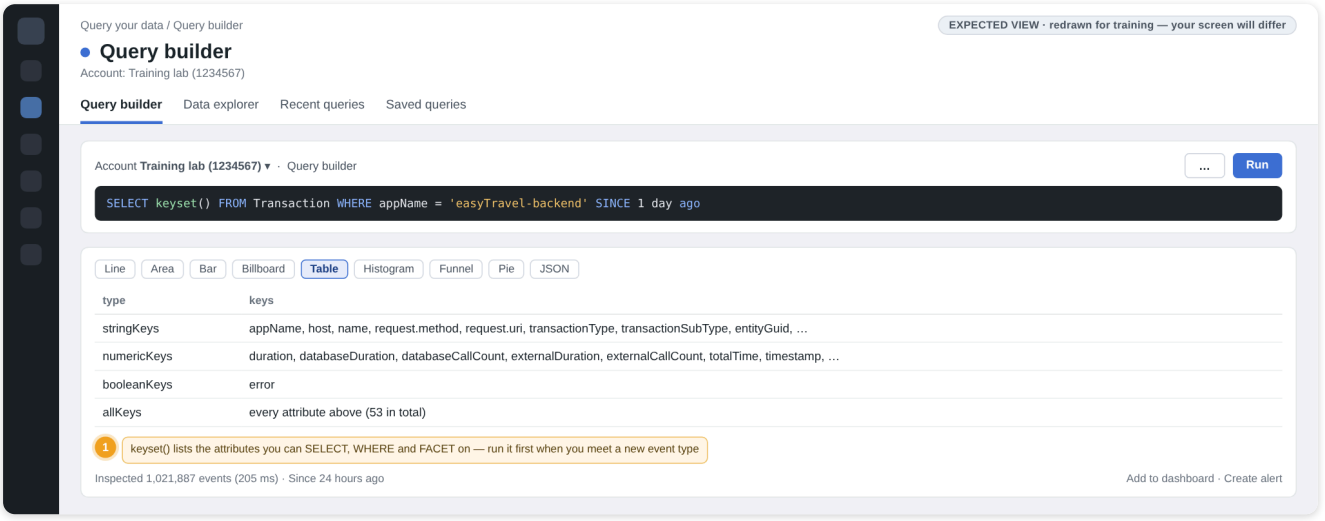
4. Function reference

NRQL has two kinds of functions. **Aggregator functions** turn many rows into one value (per facet, per time bucket). **Row functions** (non-aggregating) transform a value on each row and can be used in `SELECT`, `WHERE`, `FACET` and inside aggregators.

4.1 Aggregator functions

Function	Returns	Example
`count(*)`	Number of rows (or of non-null values with `count(attr)`)	`SELECT count(*) FROM PageView`
`sum(attr)`	Total	`SELECT sum(databaseCallCount) FROM Transaction`
`average(attr)`	Mean	`SELECT average(duration) FROM Transaction`
`min(attr)` / `max(attr)`	Smallest / largest	`SELECT max(duration) FROM Transaction`
`median(attr)`	50th percentile	`SELECT median(duration) FROM Transaction`
`percentile(attr, p [, p …])`	One or more percentiles	`SELECT percentile(duration, 50, 95, 99) FROM Transaction`
`stddev(attr)` / `stdvar(attr)`	Standard deviation / variance	`SELECT stddev(duration) FROM Transaction`
`uniqueCount(attr [, attr …])`	Number of distinct values (exact up to 256 by default, `precision:` up to 50,000)	`SELECT uniqueCount(session) FROM PageView`
`uniques(attr [, limit])`	The distinct values themselves (default 1,000, max 10,000)	`SELECT uniques(appName) FROM Transaction`
`latest(attr)` / `earliest(attr)`	Value from the newest / oldest row	`SELECT latest(appVersion) FROM Transaction`
`rate(function, interval)`	Aggregator scaled to a rate	`SELECT rate(count(*), 1 minute) FROM Transaction`
`percentage(function, WHERE cond)`	Share of rows matching a condition	`SELECT percentage(count(*), WHERE error IS true) FROM Transaction`
`filter(function, WHERE cond)`	Aggregator over a subset; several in one query	`SELECT filter(count(*), WHERE httpResponseCode = '500') FROM Transaction`
`apdex(attr, t: seconds)`	Apdex score and its satisfied / tolerating / frustrated counts	`SELECT apdex(duration, t: 0.5) FROM Transaction`
`histogram(attr, width:, buckets:, start:)`	Distribution in buckets (1–500 buckets)	`SELECT histogram(duration, width: 0.1, buckets: 20) FROM Transaction`

Function	Returns	Example
`funnel(attr, WHERE step1, WHERE step2 …)`	How many `attr` values (e.g. sessions) pass each step in order	see section 5
`cdfPercentage(attr, threshold [, …])`	% of values at or below each threshold (up to 10)	`SELECT cdfPercentage(duration, 0.1, 0.5, 1) FROM Transaction`
`derivative(attr [, interval])`	Rate of change of a value over time	`SELECT derivative(diskUsedBytes, 1 hour) FROM StorageSample`
`latestrate(attr, interval)`	Rate of change between the last two values	`SELECT latestrate(networkReceiveBytes, 1 second) FROM NetworkSample`
`predictLinear(attr [, interval])`	Linear forecast of where a value will be	`SELECT predictLinear(diskUsedPercent, 1 day) FROM StorageSample`
`bucketPercentile(attr [, p …])`	Percentiles from Prometheus-style histogram buckets	`SELECT bucketPercentile(http_request_duration_seconds_bucket, 95) FROM Metric`
`keyset()`	The attributes present in the data	`SELECT keyset() FROM Transaction`
`cardinality([metric])`	Number of distinct dimension combinations of metrics	`SELECT cardinality() FROM Metric FACET metricName`
`bytecountestimate()`	Estimated stored bytes (for cost questions)	`SELECT bytecountestimate()/1e9 AS 'GB' FROM Log FACET entity.name`



Expected view — keyset() lists the attributes you can use on an event type

4.2 Math

Operators `+ - * /` and these functions work on numbers and on aggregated results:

Function	Meaning
`abs(n)`	Absolute value

Function	Meaning
`round(n)` · `floor(n)` · `ceil(n)`	Nearest, down, up to an integer
`clamp_min(n, lo)` · `clamp_max(n, hi)`	Lower / upper bound
`pow(n, m)` · `sqrt(n)` · `exp(n)`	Power, square root, e^n
`ln(n)` · `log2(n)` · `log10(n)` · `log(n, base)`	Logarithms
`mod(a, b)`	Remainder

```
SELECT round(average(duration) * 1000) AS 'avg ms',
       clamp_max(percentile(duration, 99), 10) AS 'p99 (s, capped at 10)'
FROM Transaction WHERE appName = 'easyTravel-backend'
```

4.3 String functions

Function	Meaning	Example
`lower(s)` · `upper(s)`	Change case	`FACET lower(request.method)`
`length(s)`	Length of a string (or array)	`WHERE length(message) > 2000`
`substring(s, start [, length])`	Part of a string (0-based start)	`FACET substring(host, 0, 12)`
`position(s, sub [, occurrence])`	Where `sub` appears (–1 if absent)	`WHERE position(request.uri, '/services/') = 0`
`concat(a, b, …)`	Join values into one string (up to 20 arguments)	`FACET concat(appName, ' · ', host)`
`capture(s, r'regex')`	Values of RE2 named groups `(?P<name>…)`	see section 5
`aparse(s, 'pattern')`	Anchor-based extraction: `*` captures, `%` skips	see section 5
`jparse(s [, path])`	Parse a JSON string; optional path such as `'$.order.id'`	`SELECT jparse(payload, '$.order.id') FROM Log`
`encode(s, 'base64')` · `decode(s, 'base64')`	Base64 (also `base64url`, `base64mime`)	`SELECT decode(token, 'base64') FROM Log`

4.4 Time functions

Function	Returns	Example
`hourOf(timestamp)`	`0` – `23` hour label	`FACET hourOf(timestamp)`
`weekdayOf(timestamp)`	`Monday` … `Sunday`	`FACET weekdayOf(timestamp)`
`dateOf(timestamp)`	Calendar date	`FACET dateOf(timestamp) SINCE 1 week ago`

Function	Returns	Example
`minuteOf()` · `dayOfMonthOf()` · `weekOf()` · `monthOf()` · `quarterOf()` · `yearOf()`	Other calendar parts	`FACET monthOf(timestamp) SINCE 6 months ago`
`toDatetime(epochMillis [, pattern])`	Readable date string	`SELECT toDatetime(timestamp, 'yyyy-MM-dd HH:mm:ss')`
`toTimestamp(string [, pattern])`	Epoch milliseconds from a date string	`WHERE timestamp > toTimestamp('2026-10-27 00:00:00')`
`aggregationendtime()`	End time of each bucket or the whole window	`SELECT aggregationendtime(), count(*) … TIMESERIES`

All time functions follow `WITH TIMEZONE` when it is set.

4.5 Type conversion and conditional

Function	Meaning	Example
`numeric(s)`	String → number (null if not numeric)	`average(numeric(responseSize))`
`string(v [, precision: n])`	Number or boolean → string	`FACET string(httpResponseCode)`
`boolean(s)`	`'true'` / `'false'` → boolean	`WHERE boolean(isCached) IS true`
`if(cond, then [, else])`	Value chosen by a condition	`FACET if(duration > 1, 'slow', 'ok')`
`convert(v, 'unit', 'unit')`	Unit conversion	`SELECT average(convert(duration, 's', 'ms'))`

4.6 Utility functions

Function	Meaning
`eventType()`	Data type of each row — useful with `FROM A, B` and `FACET eventType()`
`accountId()`	Account of each row (for cross-account dashboards)
`getField(attr, field)` or `attr[field]`	A field of a structured value, e.g. a metric's `max` or an array element (index from 0)
`buckets(attr, ceiling [, n])`	Numeric facet ranges: `FACET buckets(duration, 2, 10)`
`cidrAddress(ip, prefix)`	Group IPv4 addresses into a network: `FACET cidrAddress(remoteAddress, 24)`
`lookup(table)`	Rows from an uploaded lookup table (CSV) to enrich results

Function	Meaning
`dimensions()`	All dimensions of a metric, for `FACET dimensions()`
`blob(attr)`	Read large text stored as a blob (max `LIMIT` 20; not in `WHERE`, `FACET` or `TIMESERIES`)
`contains(array, value)` · `length(array)`	Test or count array elements

> Function availability grows over time. When a function here is not recognised in your account, check the *NRQL syntax, clauses and functions* reference linked in the sources.

5. Advanced techniques

5.1 Several filtered values in one query — filter()

`filter()` applies its own `WHERE` to one aggregator, so a single query can return a whole status-code breakdown or a before/after pair:

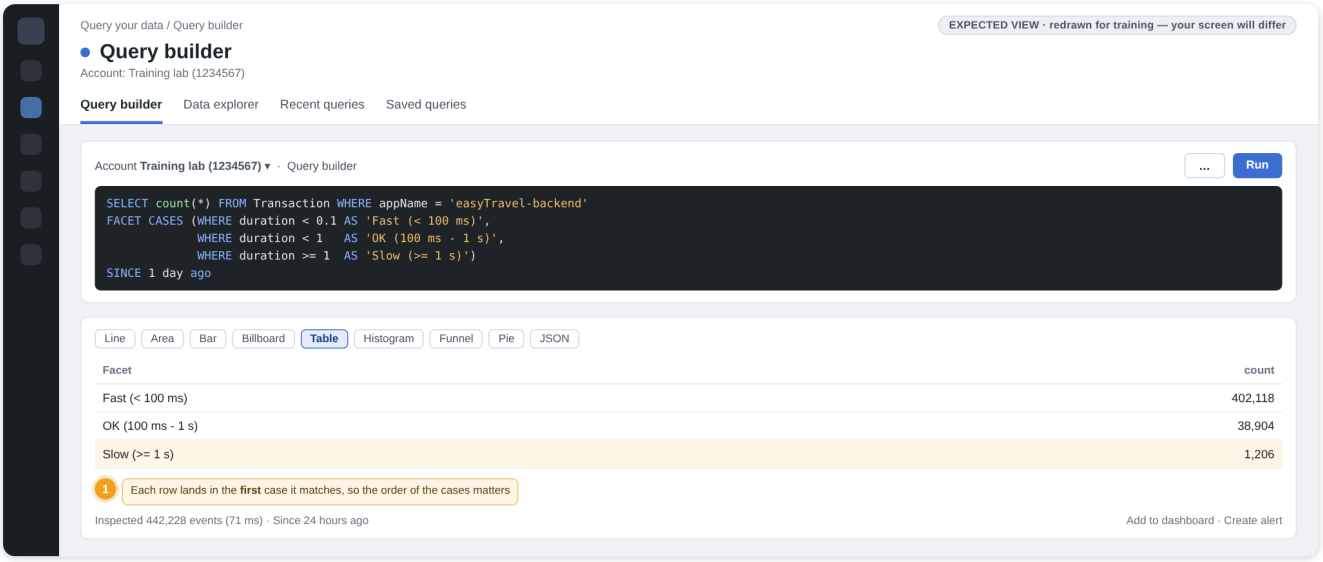
```
SELECT filter(count(*), WHERE httpResponseCode LIKE '2%') AS '2xx',
       filter(count(*), WHERE httpResponseCode LIKE '4%') AS '4xx',
       filter(count(*), WHERE httpResponseCode LIKE '5%') AS '5xx',
       percentage(count(*), WHERE httpResponseCode LIKE '5%') AS '5xx %'
FROM Transaction WHERE appName = 'easyTravel-frontend' SINCE 1 day ago TIMESERIES 1 hour
```

5.2 Rates

```
-- requests per minute, averaged over each 5-minute bucket
SELECT rate(count(*), 1 minute) AS 'rpm' FROM Transaction
WHERE appName = 'easyTravel-backend' TIMESERIES 5 minutes SINCE 3 hours ago
```

5.3 Custom groups — FACET CASES and buckets()

```
SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-backend'
FACET CASES (WHERE duration < 0.1 AS 'Fast (< 100 ms)',
             WHERE duration < 1   AS 'OK (100 ms - 1 s)',
             WHERE duration >= 1  AS 'Slow (>= 1 s)')
SINCE 1 day ago
```

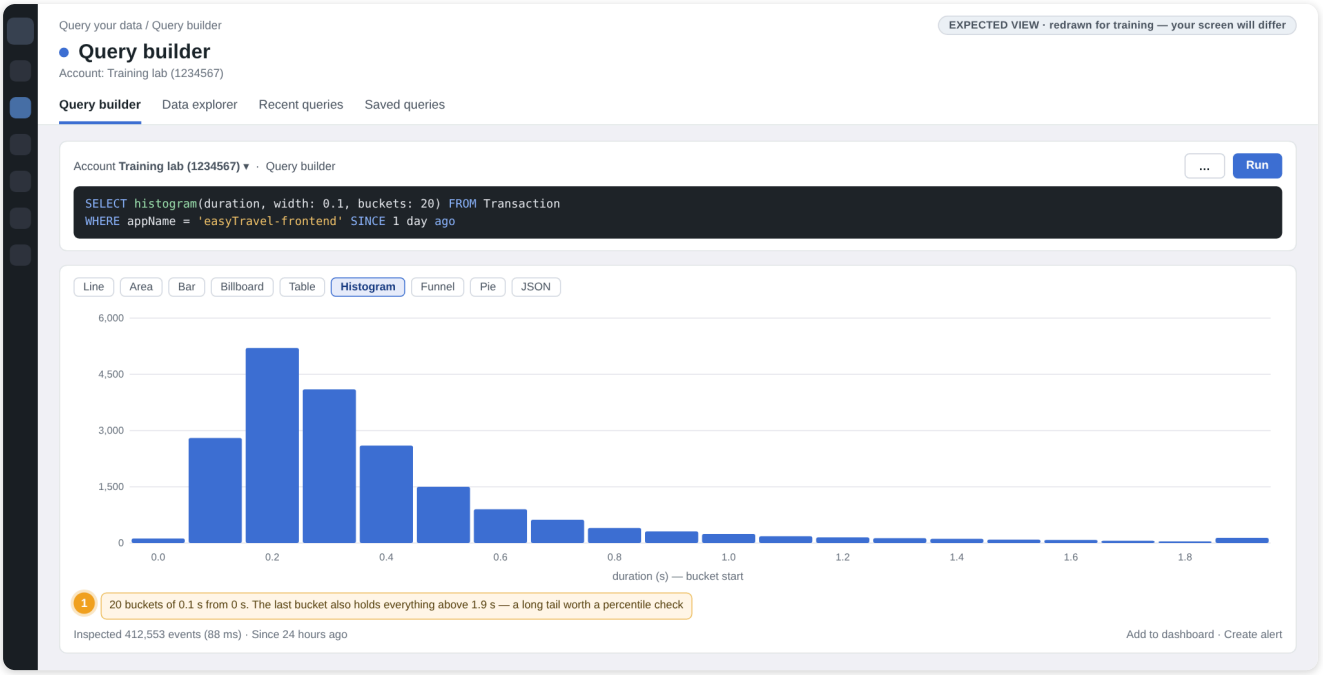


Expected view — FACET CASES: three custom latency bands

A row lands in the **first** case it matches. Add `OR attribute` after the parenthesis to facet the rows that match no case by that attribute. For equal-width numeric ranges, `FACET buckets(duration, 2, 10)` makes ten 0.2-second bands from 0 to 2 s.

5.4 Distributions — histogram() and percentiles

```
SELECT histogram(duration, width: 0.1, buckets: 20) FROM Transaction
WHERE appName = 'easyTravel-frontend' SINCE 1 day ago
```

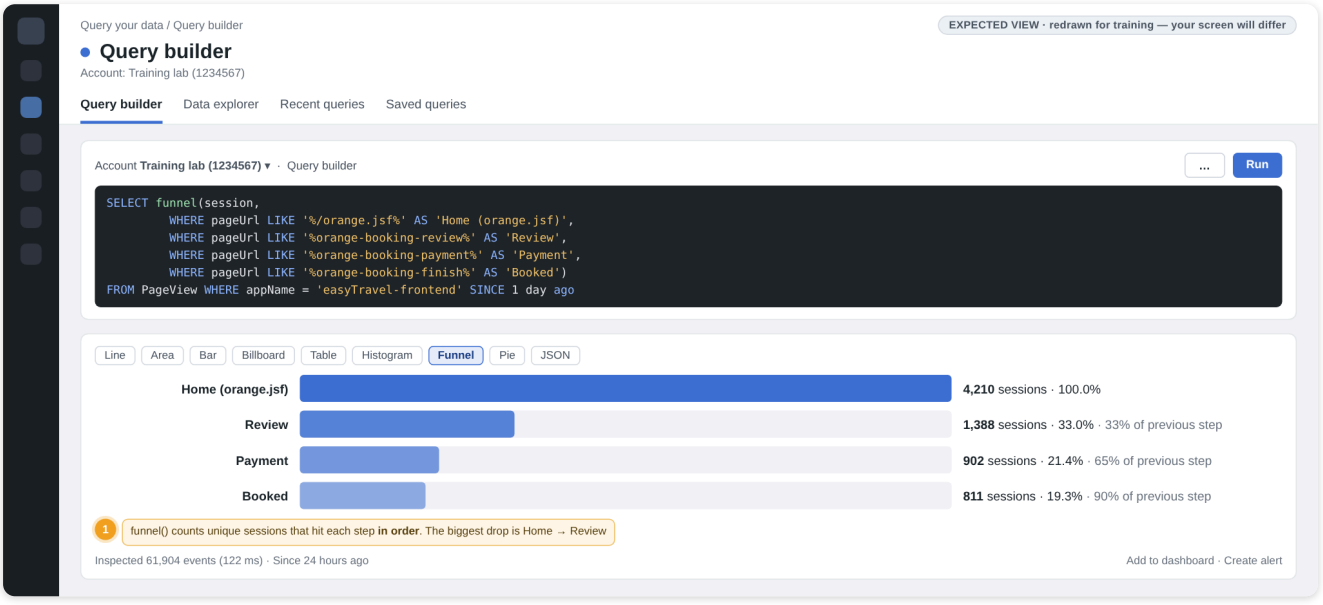


Expected view — histogram(): most requests take 0.1–0.3 s, with a long tail

Averages hide tails; histograms and percentiles show them. Pair the histogram with `percentile(duration, 50, 90, 99)` and `cdfPercentage(duration, 0.5, 1, 2)` ("what share finished within 0.5 s, 1 s, 2 s?").

5.5 Funnels — funnel()

```
SELECT funnel(session,
         WHERE pageUrl LIKE '%/orange.jsf%' AS 'Home (orange.jsf)',
         WHERE pageUrl LIKE '%orange-booking-review%' AS 'Review',
         WHERE pageUrl LIKE '%orange-booking-payment%' AS 'Payment',
         WHERE pageUrl LIKE '%orange-booking-finish%' AS 'Booked')
FROM PageView WHERE appName = 'easyTravel-frontend' SINCE 1 day ago
```



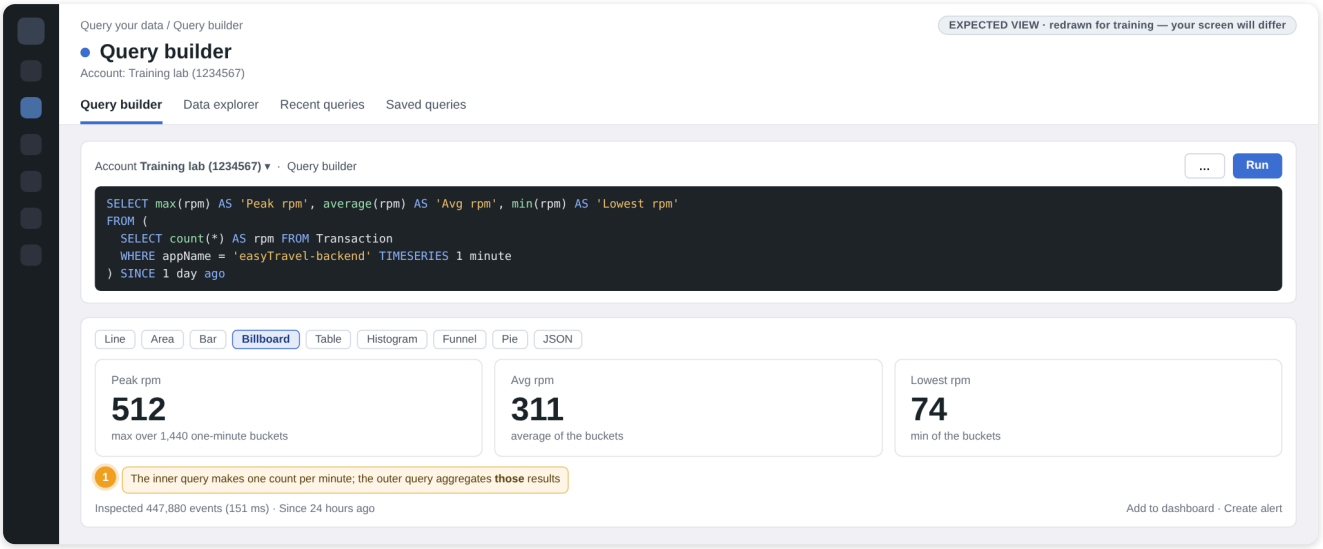
Expected view — funnel(): unique sessions at each booking step

The first argument is what you count (sessions, users); each `WHERE` is a step that must happen after the previous one. Browser data (`PageView`, `PageAction`) is the usual source.

5.6 Nested aggregation — FROM (SELECT …)

Aggregate the result of another aggregation: the peak per-minute rate, the average of hourly maximums, hosts whose average CPU crossed a threshold.

```
SELECT max(rpm) AS 'Peak rpm', average(rpm) AS 'Avg rpm', min(rpm) AS 'Lowest rpm'
FROM (
  SELECT count(*) AS rpm FROM Transaction
  WHERE appName = 'easyTravel-backend' TIMESERIES 1 minute
) SINCE 1 day ago
```



Expected view — nested aggregation: max, average and min of 1,440 one-minute counts

```
-- Hosts whose 1-minute average CPU was above 90 %
SELECT hostname, cpu FROM (
  SELECT average(cpuPercent) AS cpu FROM SystemSample
  FACET hostname TIMESERIES 1 minute
) WHERE cpu > 90 SINCE 1 hour ago
```

Rules: `SINCE`, `UNTIL` and `COMPARE WITH` go on the **outer** query only; name inner results with `AS` (otherwise refer to them by function name, e.g. `count`).

5.7 Subqueries in WHERE and SELECT

```
-- Scalar: requests slower than the overall average
SELECT count(*) FROM Transaction
WHERE duration > (SELECT average(duration) FROM Transaction) SINCE 1 hour ago

-- Delta against last week's average, as a time series
FROM Transaction SELECT average(duration) -
  (FROM Transaction SELECT average(duration) SINCE 7 days ago)
TIMESERIES SINCE 1 day ago

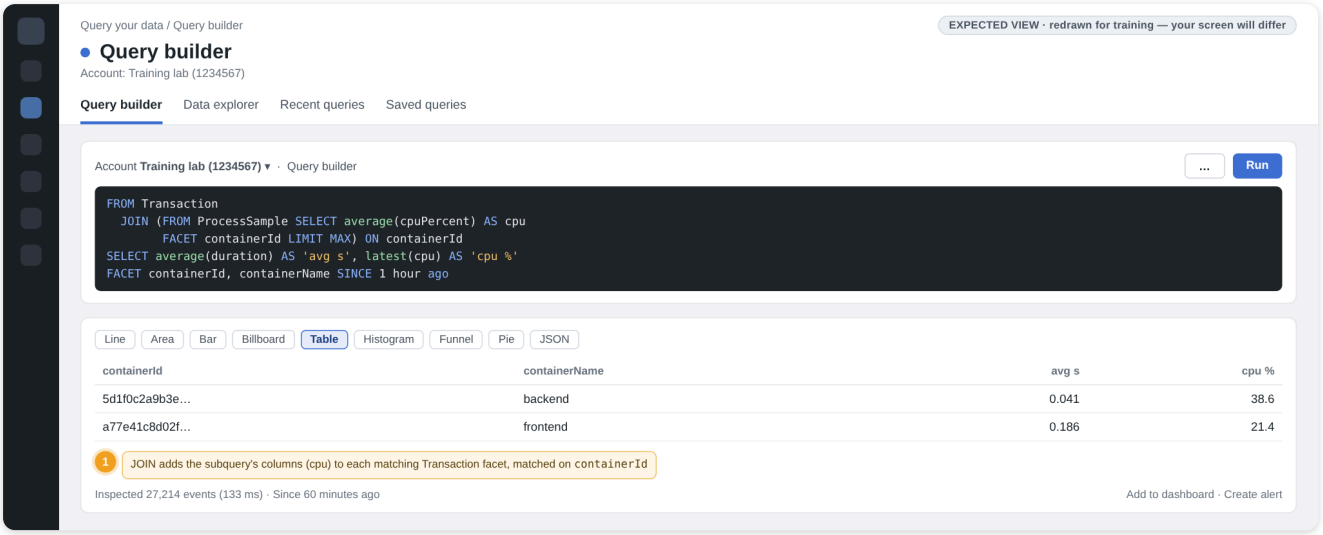
-- IN list: errors only for transactions that had a request slower than 2 s
SELECT count(*) FROM TransactionError WHERE transactionName IN
```

```
  (FROM Transaction SELECT uniques(name) WHERE duration > 2)
FACET transactionName SINCE 1 hour ago
```

Rules: at most three subqueries per query; a subquery returns at most 5,000 values; no `TIMESERIES` or `COMPARE WITH` inside a subquery; subqueries cannot see the outer query's rows; not supported in alert conditions.

5.8 Joins — JOIN (subquery) ON key

```
FROM Transaction
  JOIN (FROM ProcessSample SELECT average(cpuPercent) AS cpu
        FACET containerId LIMIT MAX) ON containerId
SELECT average(duration) AS 'avg s', latest(cpu) AS 'cpu %'
FACET containerId, containerName SINCE 1 hour ago
```



Expected view — JOIN: transaction latency next to container CPU, matched on containerId

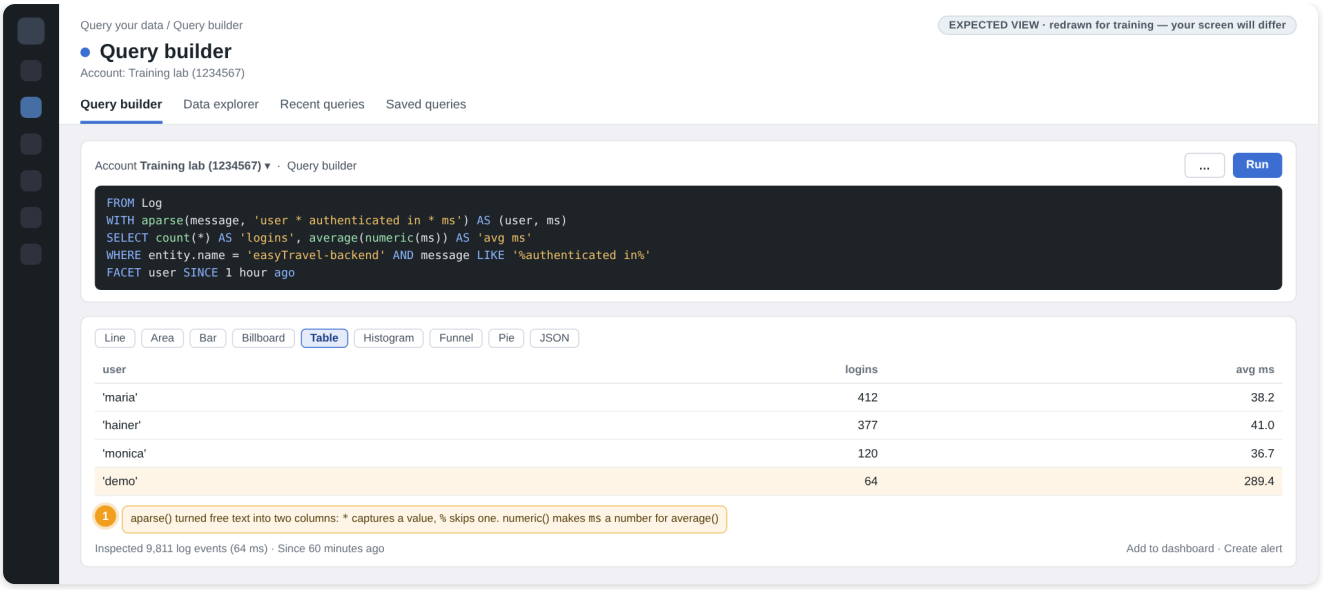
Rules: `JOIN` comes straight after `FROM`; `INNER` (default) or `LEFT`; `ON` is equality only and both keys must have the same type; the subquery is faceted by the key, defaults to `LIMIT 10` (set `LIMIT MAX`), cannot use `SELECT *` or `TIMESERIES`, and each key may match at most 100 subquery rows. Not available in alert conditions.

5.9 Extracting values from text — aparse() and capture()

Log messages and URLs often hide the value you need inside free text.

`aparse()` uses a simple pattern: `*` captures a value, `%` skips text, everything else must match literally. Combine it with `WITH … AS` to name the captures:

```
FROM Log
WITH aparse(message, 'user * authenticated in * ms') AS (user, ms)
SELECT count(*) AS 'logins', average(numeric(ms)) AS 'avg ms'
WHERE entity.name = 'easyTravel-backend' AND message LIKE '%authenticated in%'
FACET user SINCE 1 hour ago
```



Expected view — aparse(): user and duration pulled out of a log message

`capture()` uses an RE2 regular expression with a named group and returns that group's value:

```
-- service name from a URI such as /services/BookingService/storeBooking
SELECT count(*) FROM Transaction
FACET capture(request.uri, r'/services/(?P<service>[A-Za-z]+)Service.*')
SINCE 1 hour ago

-- numeric value from a message, then aggregate it
SELECT average(numeric(capture(message, r'.*took (?P<ms>\d+) ms.*'))) AS 'avg ms'
FROM Log WHERE message LIKE '%took%' SINCE 1 hour ago
```

`capture()` must match the **whole** value (hence the `.*` at both ends) and returns null when it does not. Up to five `capture()` calls per query.

`jparse()` reads JSON held in a string attribute: `SELECT jparse(body, '$.customer.tier') FROM Log`.

5.10 Comparing data types side by side

```
SELECT count(*) FROM Transaction, TransactionError, Log
WHERE appName = 'easyTravel-backend' OR entity.name = 'easyTravel-backend'
FACET eventType() TIMESERIES 10 minutes SINCE 6 hours ago
```

6. Extraction query cookbook

Ready-to-run queries grouped by data source. Change the application, host and attribute names to yours; if an attribute returns nothing, check its exact name with `SELECT keyset() FROM <Type>`.

6.1 Discovery — what data do I have?

```
SHOW EVENT TYPES SINCE 1 week ago
SELECT keyset() FROM Transaction SINCE 1 day ago
SELECT uniques(appName) FROM Transaction SINCE 1 day ago
SELECT count(*) FROM Transaction, Span, Log, PageView FACET eventType() SINCE 1 day ago
SELECT uniques(metricName) FROM Metric WHERE metricName LIKE 'apm.%' SINCE 1 hour ago
```

6.2 APM — transactions

```
-- Golden signals for one service
SELECT count(*) AS 'throughput', average(duration) AS 'avg s',
       percentile(duration, 95) AS 'p95 s',
       percentage(count(*), WHERE error IS true) AS 'error %',
       apdex(duration, t: 0.5) AS 'apdex'
FROM Transaction WHERE appName = 'easyTravel-backend' SINCE 1 hour ago

-- Most time-consuming transactions (share of total time)
SELECT sum(duration) AS 'total s', count(*), average(duration)
FROM Transaction WHERE appName = 'easyTravel-backend' FACET name SINCE 1 day ago LIMIT 20

-- Where the time goes: database vs external vs code
SELECT average(databaseDuration) AS 'db', average(externalDuration) AS 'external',
       average(duration - databaseDuration - externalDuration) AS 'other'
FROM Transaction WHERE appName = 'easyTravel-backend' FACET name SINCE 1 hour ago

-- N+1 suspects: many database calls per request
SELECT average(databaseCallCount), max(databaseCallCount) FROM Transaction
WHERE appName = 'easyTravel-backend' FACET name, request.uri SINCE 1 hour ago

-- Background (non-web) jobs
SELECT count(*), average(duration) FROM Transaction
WHERE transactionType = 'Other' FACET appName, name SINCE 1 day ago

-- Throughput per host (instances behind a load balancer)
SELECT rate(count(*), 1 minute) FROM Transaction
WHERE appName = 'easyTravel-frontend' FACET host TIMESERIES SINCE 3 hours ago
```

6.3 Errors

```
-- Top errors by class and message
SELECT count(*) FROM TransactionError WHERE appName = 'easyTravel-backend'
FACET error.class, error.message SINCE 1 day ago LIMIT 20

-- Error rate over time, by application
SELECT percentage(count(*), WHERE error IS true) FROM Transaction
WHERE appName LIKE 'easyTravel-%' FACET appName TIMESERIES SINCE 1 day ago

-- HTTP status codes
SELECT count(*) FROM Transaction WHERE appName = 'easyTravel-frontend'
FACET http.statusCode SINCE 1 day ago
```

```
-- Raw error rows to export
SELECT timestamp, appName, transactionName, error.class, error.message, host
FROM TransactionError SINCE 1 day ago LIMIT MAX
```

6.4 Distributed tracing — spans

```
-- Slowest downstream calls (database and HTTP clients)
SELECT average(duration.ms), percentile(duration.ms, 95), count(*) FROM Span
WHERE span.kind = 'client' FACET appName, name SINCE 1 hour ago

-- Failing spans by service and error
SELECT count(*) FROM Span WHERE error.class IS NOT NULL
FACET appName, name, error.class SINCE 1 hour ago

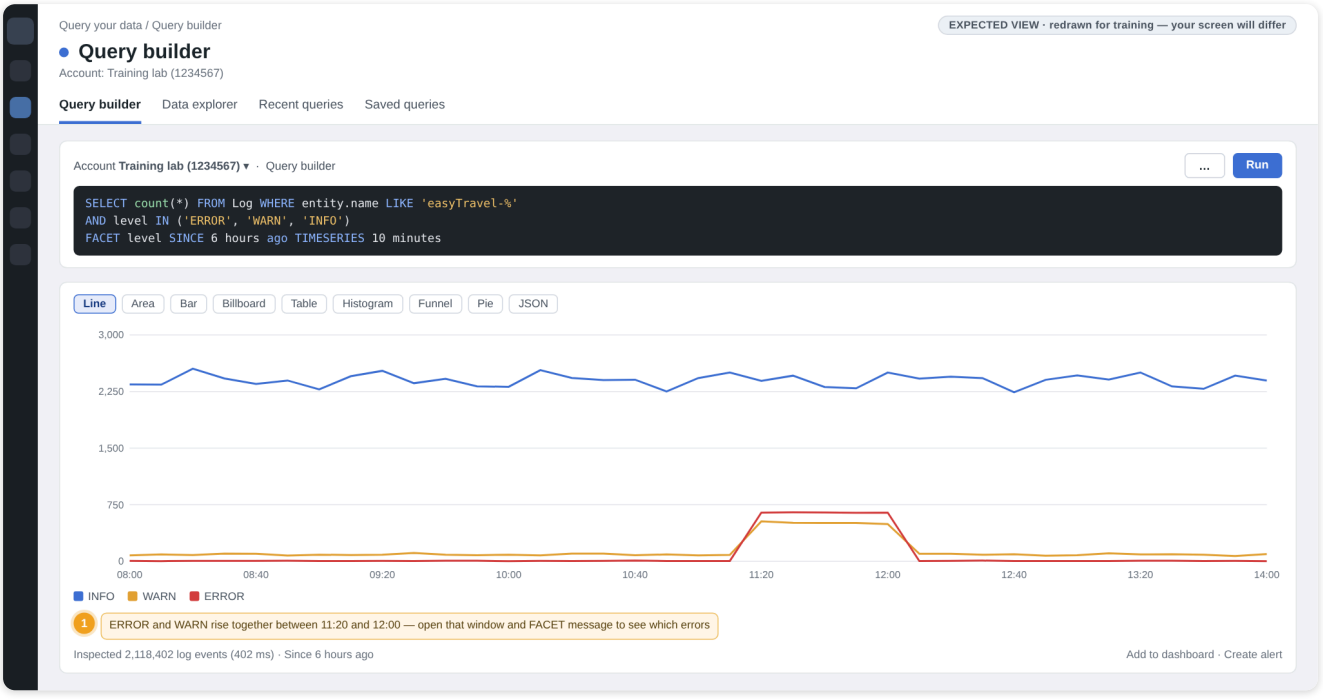
-- Every span of one trace, in order
SELECT timestamp, appName, name, duration.ms, parentId, id, error.class FROM Span
WHERE trace.id = '<trace id>' ORDER BY timestamp LIMIT MAX

-- Datastore operations by database and collection/table
SELECT count(*), average(duration.ms) FROM Span WHERE category = 'datastore'
FACET db.system, db.collection, db.operation SINCE 1 hour ago

-- Which services call which (edges)
SELECT count(*) FROM Span WHERE span.kind = 'client' AND category = 'http'
FACET appName, http.url SINCE 1 hour ago
```

6.5 Logs

```
-- Log volume by level
SELECT count(*) FROM Log WHERE entity.name LIKE 'easyTravel-%'
AND level IN ('ERROR', 'WARN', 'INFO')
FACET level SINCE 6 hours ago TIMESERIES 10 minutes
```



Expected view — log volume by level: ERROR and WARN rise together

```
-- Top error messages
SELECT count(*) FROM Log WHERE level = 'ERROR' FACET entity.name, message SINCE 1 hour ago
LIMIT 20

-- Lines of one request (logs in context)
SELECT timestamp, entity.name, level, message FROM Log
WHERE trace.id = '<trace id>' ORDER BY timestamp

-- Free-text search (case-insensitive)
SELECT timestamp, hostname, message FROM Log WHERE message LIKE '%timeout%' SINCE 30 minutes
ago LIMIT 100

-- Which sources send the most log data (cost)
SELECT bytecountestimate()/1e9 AS 'GB' FROM Log FACET entity.name, hostname SINCE 1 day ago
```

6.6 Browser (real user monitoring)

```
-- Page load time by page
SELECT average(duration), percentile(duration, 75) FROM PageView
WHERE appName = 'easyTravel-frontend' FACET pageUrl SINCE 1 day ago LIMIT 20

-- Core Web Vitals at the 75th percentile
SELECT percentile(largestContentfulPaint, 75) AS 'LCP (s)',
       percentile(interactionToNextPaint, 75) AS 'INP (ms)',
       percentile(cumulativeLayoutShift, 75) AS 'CLS'
FROM PageViewTiming WHERE appName = 'easyTravel-frontend' SINCE 1 day ago

-- JavaScript errors
SELECT count(*) FROM JavaScriptError WHERE appName = 'easyTravel-frontend'
FACET errorClass, errorMessage SINCE 1 day ago

-- Users by country and browser
SELECT uniqueCount(session) FROM PageView FACET countryCode, userAgentName SINCE 1 week ago
```

6.7 Infrastructure

```
-- CPU and memory per host
SELECT average(cpuPercent) AS 'cpu %', average(memoryUsedPercent) AS 'mem %'
FROM SystemSample FACET hostname SINCE 1 hour ago

-- Disks filling up
SELECT latest(diskUsedPercent) FROM StorageSample
WHERE diskUsedPercent > 80 FACET hostname, mountPoint SINCE 30 minutes ago

-- Top processes by CPU
SELECT average(cpuPercent), average(memoryResidentSizeBytes)/1e6 AS 'RSS MB'
FROM ProcessSample FACET hostname, processDisplayName SINCE 30 minutes ago LIMIT 20

-- Network throughput per interface
SELECT average(receiveBytesPerSecond), average(transmitBytesPerSecond)
FROM NetworkSample FACET hostname, interfaceName TIMESERIES SINCE 1 hour ago

-- Hosts that stopped reporting in the last 10 minutes
SELECT latest(timestamp) FROM SystemSample FACET hostname SINCE 1 day ago LIMIT MAX
-- (sort by latest(timestamp); anything older than ~10 minutes is silent)
```

6.8 Kubernetes

```
-- Pods that are not running
SELECT latest(status) FROM K8sPodSample WHERE status != 'Running'
FACET clusterName, namespaceName, podName SINCE 10 minutes ago

-- Container restarts
SELECT max(restartCount) FROM K8sContainerSample
FACET namespaceName, podName, containerName SINCE 1 hour ago LIMIT 20

-- CPU and memory per namespace
SELECT sum(cpuUsedCores), sum(memoryWorkingSetBytes)/1e9 AS 'GB'
FROM K8sContainerSample FACET namespaceName TIMESERIES SINCE 1 hour ago
```

6.9 Synthetic monitoring

```
-- Success rate and duration per monitor and location
SELECT percentage(count(*), WHERE result = 'SUCCESS') AS 'success %',
       average(duration) AS 'avg ms'
FROM SyntheticCheck FACET monitorName, locationLabel SINCE 1 day ago

-- Failed checks with the error
SELECT timestamp, monitorName, locationLabel, error FROM SyntheticCheck
WHERE result = 'FAILED' SINCE 1 day ago LIMIT 100
```

6.10 Metrics

```
-- APM metrics (kept longer than events; good for long time ranges)
SELECT average(apm.service.transaction.duration) * 1000 AS 'avg ms'
FROM Metric WHERE appName = 'easyTravel-backend' FACET transactionName
SINCE 30 days ago TIMESERIES 1 day

-- Any dimensional metric: list names, then query one
SELECT uniques(metricName) FROM Metric SINCE 1 hour ago
SELECT average(cpu.utilization) FROM Metric FACET host.name TIMESERIES SINCE 1 hour ago

-- Prometheus: targets that are down (aggregators cannot go in WHERE,
-- so filter the aggregated value in an outer query)
SELECT instance, job, up FROM (
  SELECT latest(up) AS up FROM Metric FACET instance, job LIMIT MAX
) WHERE up = 0 SINCE 10 minutes ago
```

6.11 Alerts and incidents

```
-- Incidents opened in the last 7 days, by policy and condition
SELECT count(*) FROM NrAiIncident WHERE event = 'open'
FACET policyName, conditionName SINCE 7 days ago LIMIT 50

-- Noisiest entities
SELECT count(*) FROM NrAiIncident WHERE event = 'open'
FACET entity.name SINCE 30 days ago LIMIT 20

-- Incident list to export
SELECT timestamp, priority, title, policyName, conditionName, entity.name
FROM NrAiIncident WHERE event = 'open' SINCE 7 days ago LIMIT MAX
```

6.12 Deployments and changes

```
-- Deployment markers (legacy markers and many change-tracking deployments)
SELECT timestamp, appName, version, revision, description, user
FROM Deployment SINCE 30 days ago LIMIT MAX

-- Before vs after a release
SELECT average(duration), percentage(count(*), WHERE error IS true) FROM Transaction
WHERE appName = 'easyTravel-backend' SINCE 2 hours ago COMPARE WITH 1 day ago TIMESERIES
```

6.13 Usage, cost and audit

```
-- Data ingested per source this month (GB)
SELECT sum(GigabytesIngested) FROM NrConsumption
WHERE productLine = 'DataPlatform' FACET usageMetric SINCE this month

-- Ingest trend by day
SELECT sum(GigabytesIngested) FROM NrConsumption
WHERE productLine = 'DataPlatform' SINCE 30 days ago TIMESERIES 1 day

-- Who changed what in the account
SELECT timestamp, actorEmail, actionIdentifier, targetType, description
FROM NrAuditEvent SINCE 7 days ago LIMIT MAX

-- Data that New Relic rejected (limits, bad attributes)
SELECT count(*) FROM NrIntegrationError FACET newRelicFeature, category, message SINCE 1 day
ago

-- Your own query and API limits
FROM Metric SELECT latest(newrelic.resourceConsumption.limitValue)
WHERE limitTimeInterval = '1 minute' FACET limitName LIMIT MAX
```

> Attribute names on platform events (`NrAiIncident`, `NrConsumption`, `NrAuditEvent`) can change between releases. Run `keyset()` on them once before relying on a query.

7. Getting data out

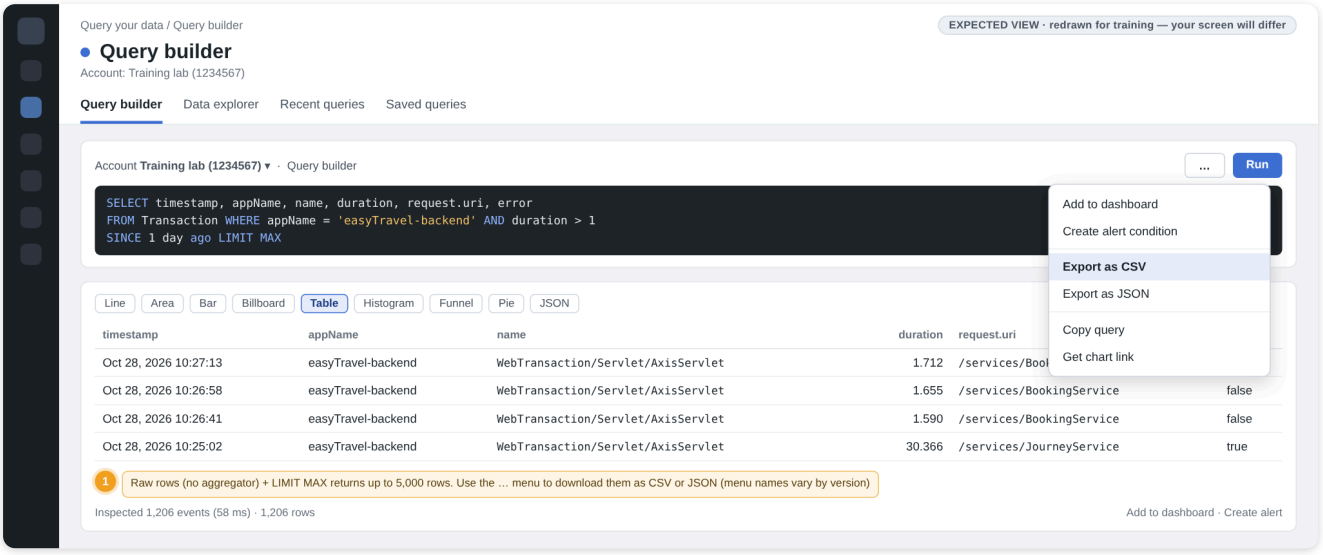
Method	Best for	Row limit per query
Query builder → *Export as CSV / JSON*	One-off downloads	5,000 (`LIMIT MAX`)
Dashboards → widget … menu	Sharing a live view; CSV of one widget	5,000
NerdGraph API (GraphQL over HTTPS)	Scripts, scheduled reports, ETL	5,000 per call — slice the time window for more
New Relic CLI `newrelic nrql query`	Shell scripts and quick checks	5,000

Whatever the method, a query returns **at most 5,000 rows or facets**. For larger exports, run the same query over consecutive time slices (7.5).

7.1 From the query builder

Run a raw-row query (no aggregator) with `LIMIT MAX`, then use the … menu next to *Run* to export:

```
SELECT timestamp, appName, name, duration, request.uri, error
FROM Transaction WHERE appName = 'easyTravel-backend' AND duration > 1
SINCE 1 day ago LIMIT MAX
```



Expected view — the query builder's … menu: export the result as CSV or JSON

CSV timestamps are exported as shown in the table; JSON keeps epoch milliseconds. Menu names differ slightly between versions of the UI.

7.2 NerdGraph API

Item	Value
Endpoint	`https://api.newrelic.com/graphql` (EU accounts: `https://api.eu.newrelic.com/graphql`)

Item	Value
Auth header	`API-Key: <user key>` (a **user** key, type *USER*, not the license key)
Query	`{ actor { account(id: <account id>) { nrql(query: "<NRQL>") { results } } } }`
Timeout	5 seconds by default; add `timeout: <seconds>` to `nrql(…)` for longer queries, or use asynchronous queries
Cross-account	`actor { nrql(accounts: [id1, id2, …], query: "…") { results } }` — up to 30 accounts
Rate limit	3,000 API queries per account per minute; repeated identical queries limited to 120 per minute (HTTP 429)

```
curl -s https://api.newrelic.com/graphql \
  -H 'Content-Type: application/json' \
  -H "API-Key: $NEW_RELIC_API_KEY" \
  -d '{"query":"{ actor { account(id: 1234567) { nrql(query: \"SELECT count(*), average(duration) FROM Transaction FACET appName SINCE 1 hour ago\") { results } } } }"}' | jq .
```

NerdGraph · run NRQL with curl — EXPECTED OUTPUT

```
trainee@nr-lab:~$ curl -s https://api.newrelic.com/graphql -H 'Content-Type: application/json'
                    -H "API-Key: $NEW_RELIC_API_KEY" -d '{"query":"{ actor { account(id:
                    1234567) { nrql(query: \"SELECT count(*), average(duration) FROM
                    Transaction FACET appName SINCE 1 hour ago\") { results } } } }"}' | jq .
{
  "data": {
    "actor": {
      "account": {
        "nrql": {
          "results": [
            {
              "appName": "easyTravel-backend",
              "average.duration": 0.0412,
              "count": 18612,
              "facet": "easyTravel-backend"
            },
            {
              "appName": "easyTravel-frontend",
              "average.duration": 0.1861,
              "count": 8402,
              "facet": "easyTravel-frontend"
            }
          ]
        }
      }
    }
  }
}
```

Expected output — NerdGraph returns NRQL results as JSON objects, one per facet

Result keys follow the function and attribute names: `count`, `average.duration`, `percentile.duration` (an object keyed by percentile), `facet` (the facet value, or an array for several facets). Ask also for

`metadata` to get the time window and facet names, or `embeddedChartUrl` / `staticChartUrl` to get a chart link.

> **Keep the user key secret.** Store it in an environment variable or a secrets manager, never in the script or in Git, and give automation its own key so it can be revoked on its own.

7.3 New Relic CLI

```
newrelic profile add --profile prod --region US --apiKey "$NEW_RELIC_API_KEY" --accountId
1234567
newrelic nrql query --accountId 1234567 \
  --query "SELECT count(*) FROM Transaction FACET appName SINCE 1 hour ago"
```

New Relic CLI · newrelic nrql query — EXPECTED OUTPUT

```
trainee@nr-lab:~$ newrelic nrql query --accountId 1234567 --query "SELECT count(*) FROM
                    Transaction FACET appName SINCE 1 hour ago"
[
  {
    "appName": "easyTravel-backend",
    "count": 18612,
    "facet": "easyTravel-backend"
  },
  {
    "appName": "easyTravel-frontend",
    "count": 8402,
    "facet": "easyTravel-frontend"
  }
]
the CLI uses the default profile's user key and region (newrelic profile list)
```

Expected output — newrelic nrql query prints the results as a JSON array

7.4 Extract to CSV with curl and jq

Passing the NRQL as a GraphQL **variable** avoids escaping quotes inside the query:

```
Q="SELECT timestamp, name, duration, request.uri FROM Transaction WHERE appName =
'easyTravel-backend' AND duration > 1 SINCE 1 day ago LIMIT MAX"

jq -n --arg q "$Q" '{query: "query($q: Nrql!) { actor { account(id: 1234567) { nrql(query:
$q, timeout: 120) { results } } } }", variables: {q: $q}}' \
  | curl -s https://api.newrelic.com/graphql -H 'Content-Type: application/json' -H "API-Key:
$NEW_RELIC_API_KEY" -d @- \
  | jq -r '.data.actor.account.nrql.results | (.[0] | keys_unsorted) as $k | ($k | @csv), (.
[] | [.[$k[]]] | @csv)' > slow_bookings.csv
```

Extract to CSV · NerdGraph + jq — EXPECTED OUTPUT

```
trainee@nr-lab:~$ Q="SELECT timestamp, name, duration, request.uri FROM Transaction WHERE
                      appName = 'easyTravel-backend' AND duration > 1 SINCE 1 day ago LIMIT MAX"
trainee@nr-lab:~$ jq -n --arg q "$Q" '{query: "query($q: Nrql!) { actor { account(id: 1234567) {
                       nrql(query: $q, timeout: 120) { results } } } }", variables: {q: $q}}' \
    | curl -s https://api.newrelic.com/graphql -H 'Content-Type: application/json' -H "API-Key:
    $NEW_RELIC_API_KEY" -d @- \
    | jq -r '.data.actor.account.nrql.results | (.[0] | keys_unsorted) as $k | ($k | @csv), (.[]
     | [.[$k[]]] | @csv)' > slow_bookings.csv
trainee@nr-lab:~$ wc -l slow_bookings.csv; head -3 slow_bookings.csv
1207 slow_bookings.csv
"duration","name","request.uri","timestamp"
1.712,"WebTransaction/Servlet/AxisServlet","/services/BookingService",1793118433000
1.655,"WebTransaction/Servlet/AxisServlet","/services/BookingService",1793118418000
header row + 1,206 data rows; timestamps are epoch milliseconds
```

Expected output — the results written to a CSV file with a header row

7.5 More than 5,000 rows — slice the time window

```
#!/usr/bin/env bash
# export-nrql.sh — export raw Transaction rows for the last 24 h, one hour per call
set -euo pipefail
ACCOUNT=1234567
STEP=$((3600 * 1000))                        # 1 hour in milliseconds
END=$(( $(date +%s) * 1000 ))
START=$(( END - 24 * STEP ))
: > transactions.ndjson
for (( t = START; t < END; t += STEP )); do
  Q="SELECT timestamp, name, duration, request.uri FROM Transaction WHERE appName =
'easyTravel-backend' SINCE $t UNTIL $(( t + STEP )) LIMIT MAX"
  jq -n --arg q "$Q" --argjson id "$ACCOUNT" \
     '{query: "query($id: Int!, $q: Nrql!) { actor { account(id: $id) { nrql(query: $q,
timeout: 120) { results } } } }", variables: {id: $id, q: $q}}' \
  | curl -s https://api.newrelic.com/graphql -H 'Content-Type: application/json' -H "API-Key:
$NEW_RELIC_API_KEY" -d @- \
  | jq -c '.data.actor.account.nrql.results[]' >> transactions.ndjson
  sleep 1                                     # stay well under the API rate limit
done
wc -l transactions.ndjson
```

If any slice returns exactly 5,000 rows it was cut off — make `STEP` smaller. Aggregated exports (counts per hour, per host) rarely need slicing: aggregate first, export the summary.

7.6 Python

```
import csv, os, requests

URL = "https://api.newrelic.com/graphql"
GQL = "query($id: Int!, $q: Nrql!) { actor { account(id: $id) { nrql(query: $q, timeout: 120)
{ results } } } }"
NRQL = "SELECT count(*), average(duration) FROM Transaction FACET appName SINCE 1 day ago"

resp = requests.post(URL, timeout=150,
                     headers={"API-Key": os.environ["NEW_RELIC_API_KEY"]},
                     json={"query": GQL, "variables": {"id": 1234567, "q": NRQL}})
resp.raise_for_status()
```

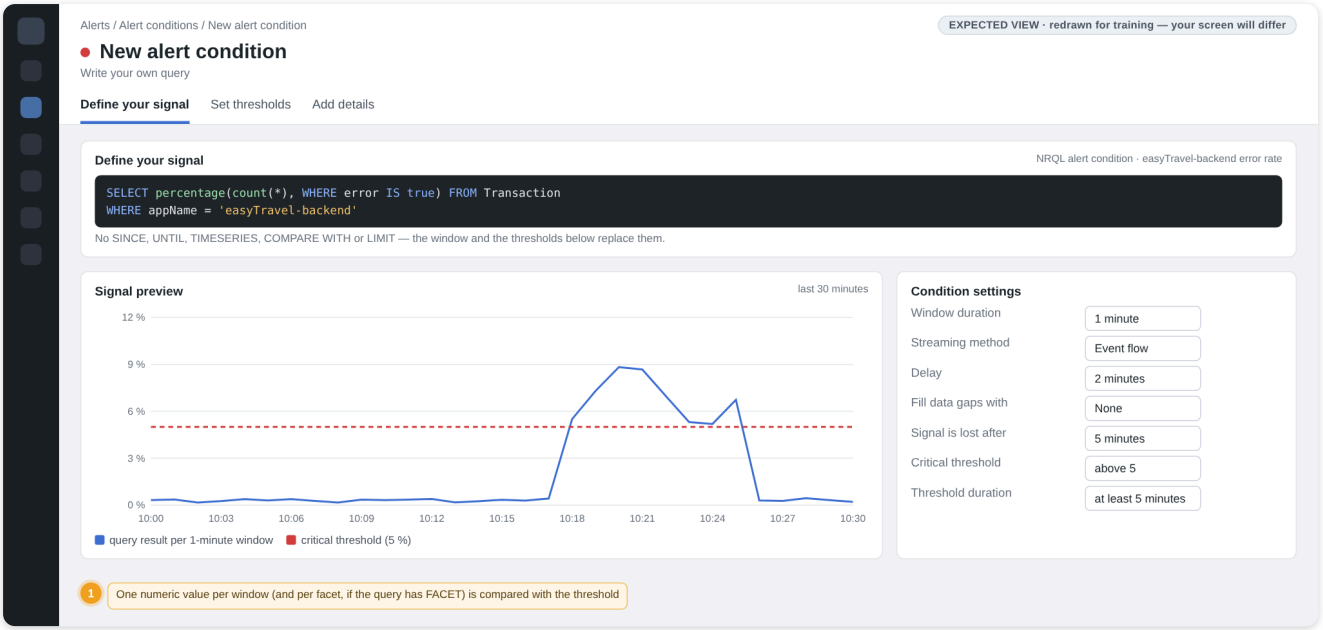
```
body = resp.json()
if body.get("errors"):
    raise SystemExit(body["errors"])
rows = body["data"]["actor"]["account"]["nrql"]["results"]

with open("by_app.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=sorted({k for r in rows for k in r}))
    writer.writeheader()
    writer.writerows(rows)
print(f"wrote {len(rows)} rows")
```

GraphQL errors (a bad NRQL query, a timeout) come back with HTTP 200 and an `errors` array — always check it, as above.

8. NRQL in alert conditions

An NRQL alert condition runs your query continuously, one **aggregation window** at a time (1 minute by default), and compares the single number it returns with a threshold. With `FACET`, each facet value is a separate signal and alerts on its own.



Expected view — an NRQL alert condition: the query, its signal and the window settings

Rules for alert queries

Clause or feature	In an alert condition
`SELECT` + `FROM` (+ `WHERE`)	Required; must return **one number** per window (per facet)
`FACET`	Allowed; every value alerts separately (no `LIMIT`; up to 20,000 values)
`SINCE` / `UNTIL`	Not allowed — the window setting replaces them
`TIMESERIES` / `SLIDE BY`	Not allowed — use *window duration* and *sliding window aggregation*
`COMPARE WITH`	Not allowed — use an **anomaly** threshold instead
`LIMIT`	Not allowed
Subqueries and `JOIN`	Not supported
Nested aggregation	Supported with conditions: the inner query must be faceted and use the same window
Functions	Use numeric aggregators: `count`, `sum`, `average`, `min`, `max`, `latest`, `percentage`, `percentile`, `uniqueCount`, `apdex`. Not `histogram()`, `cardinality()`, `bytecountestimate()`
One value	Several aggregators in one query are not supported — make one condition per value

Example alert queries

```
-- Error rate above 5 %
SELECT percentage(count(*), WHERE error IS true) FROM Transaction
WHERE appName = 'easyTravel-backend'

-- 95th percentile latency, one signal per application
SELECT percentile(duration, 95) FROM Transaction WHERE appName LIKE 'easyTravel-%' FACET
appName

-- Apdex with its own T
SELECT apdex(duration, t: 0.3) FROM Transaction WHERE appName = 'easyTravel-backend' AND name
LIKE '%Booking%'

-- Synthetic failures that return 0, not NULL, when nothing fails
SELECT filter(count(*), WHERE result = 'FAILED') FROM SyntheticCheck WHERE monitorName =
'easyTravel home'

-- Disk almost full, per host and mount
SELECT latest(diskUsedPercent) FROM StorageSample FACET hostname, mountPoint

-- Error log burst
SELECT count(*) FROM Log WHERE entity.name = 'easyTravel-backend' AND level = 'ERROR'
```

Settings that change what the query sees

Setting	What it does	Typical value
Window duration	Size of each evaluation bucket (30 s – 120 min)	1 minute
Streaming method	*Event flow* (APM, most events), *Event timer* (cloud integrations, sparse data), *Cadence* (legacy)	Event flow
Delay / timer	How long to wait for late data before closing a window	2 minutes (more for cloud metrics)
Fill data gaps	*None*, a static value (e.g. 0) or *last known value* for empty windows	None, or 0 for counts
Signal loss	Open or close an incident when the signal stops (30 s – 48 h)	5–10 minutes for heartbeats
Threshold duration	How long the value must breach before an incident opens	5 minutes

Common mistakes

- **`count(*)` returns NULL, not 0,** when no rows match — a "failures above 0" condition then never fires. Use `filter(count(*), WHERE …)` with a broader `WHERE`.
- **Copying a chart query unchanged:** `SINCE` and `TIMESERIES` are removed when you create a condition from a chart; check the remaining query still returns one value.
- **Too many facets** with `percentile()` can fail — prefer `average()` or reduce facets.
- **Sparse data with Event flow** looks like gaps; switch to *Event timer* or fill gaps.

9. Limits, performance and common errors

Limits worth knowing

Limit	Value
Rows or facets returned	Default 10 facets / 100 rows; maximum 5,000 (`LIMIT MAX`)
Attributes in `FACET`	5
`TIMESERIES MAX`	At most 366 buckets
Query run time	1 minute (Original Data option), 10 minutes (Data Plus)
NerdGraph	Default timeout 5 s (raise with `timeout:` or use async queries); 3,000 queries per account per minute; 120 identical queries per minute
Cross-account queries	Up to 30 accounts
Subqueries	Up to 3 per query, each returning at most 5,000 values
`JOIN`	Subquery `LIMIT` up to 5,000; at most 100 matches per key
`uniqueCount()`	Exact up to 256 values by default; `precision:` up to 50,000
`uniques()`	1,000 values by default, up to 10,000
`capture()`	5 per query, 16 groups each
`histogram()`	1–500 buckets
`concat()`	20 arguments, 4,096 characters

Your account's own limits are queryable: `FROM Metric SELECT latest(newrelic.resourceConsumption.limitValue) WHERE limitTimeInterval = '1 minute' FACET limitName LIMIT MAX`. How long data is kept depends on the data type and your plan — see *Administration → Data retention*.

Writing fast queries

1. **Narrow the time window first.** `SINCE 1 hour ago` before `SINCE 30 days ago`; widen once the query is right.
2. **Filter early and specifically.** `WHERE appName = '…'` on an exact value is cheaper than `LIKE '%…%'`; avoid leading wildcards and `RLIKE` on large data sets when `=` or `IN` will do.
3. **Aggregate instead of pulling rows.** Count per hour in NRQL rather than exporting every row and counting in Excel.
4. **Use metrics for long ranges.** `FROM Metric` (for example `apm.service.transaction.duration`) is pre-aggregated and fast over weeks; events are better for detail.
5. **Watch high-cardinality facets.** Facetting by `request.uri` with IDs in it, or by `trace.id`, creates thousands of groups; cap them with `LIMIT` or normalise with `capture()`.

6. **Reuse with `WITH … AS`** rather than repeating the same expression three times.

Common errors and fixes

Symptom	Likely cause	Fix
No events found / empty chart	Wrong event type or attribute case, time window too short, wrong account	`SHOW EVENT TYPES`, `keyset()`, widen `SINCE`, check the account picker
Syntax error near a value	String in double quotes, or an unquoted attribute with a dot or space	Single quotes for strings; backticks for odd attribute names
`Unknown function`	Typo, or a function not available in this context (e.g. alerts)	Check section 4 and the alert rules in section 8
Mixing aggregators and attributes	`SELECT name, count(*)`	`SELECT count(*) … FACET name`, or `latest(name)`
Aggregator in `WHERE`	`WHERE count(*) > 10`	Nested aggregation: filter in an outer query (section 5.6)
Numbers treated as text	Attribute stored as a string	`numeric(attr)`
Facet list looks incomplete	Default `LIMIT 10`	`LIMIT 100` or `LIMIT MAX`
Query timed out	Window too long, unselective filter	Narrow `SINCE`, add filters, use `Metric`, or async NerdGraph
Percentages look wrong	`percentage()` counts rows, not users	Use `uniqueCount()` in the numerator and denominator for per-user rates
Counts lower than expected on busy apps	Event sampling	Add `EXTRAPOLATE`, or use APM metrics

10. One-page cheat sheet

I want to…	Write
Count rows	`SELECT count(*) FROM Transaction`
Average, max, p95	`SELECT average(duration), max(duration), percentile(duration, 95) FROM Transaction`
Filter	`… WHERE appName = 'easyTravel-backend' AND duration > 1`
Partial match	`… WHERE name LIKE '%Booking%'`
Group by	`… FACET appName, host`
More groups	`… FACET name LIMIT 50` (max `LIMIT MAX` = 5,000)
Custom groups	`… FACET CASES (WHERE duration < 1 AS 'ok', WHERE duration >= 1 AS 'slow')`
Over time	`… TIMESERIES 5 minutes` (`AUTO` , `MAX`)
Smooth a spiky line	`… TIMESERIES 5 minutes SLIDE BY 1 minute`
Time window	`… SINCE 1 day ago UNTIL 1 hour ago` · `SINCE today` · `SINCE '2026-10-27 09:00:00'`
Compare with last week	`… SINCE 1 day ago COMPARE WITH 1 week ago`
Error rate	`SELECT percentage(count(*), WHERE error IS true) FROM Transaction`
Several filtered counts	`SELECT filter(count(*), WHERE …) AS 'a', filter(count(*), WHERE …) AS 'b'`
Per-minute rate	`SELECT rate(count(*), 1 minute) FROM Transaction`
Distinct values / count	`SELECT uniques(host)` · `SELECT uniqueCount(session)`
Latest value	`SELECT latest(diskUsedPercent) FROM StorageSample FACET hostname`
Distribution	`SELECT histogram(duration, width: 0.1, buckets: 20) FROM Transaction`
Funnel	`SELECT funnel(session, WHERE … AS 'step 1', WHERE … AS 'step 2') FROM PageView`
Aggregate an aggregate	`SELECT max(rpm) FROM (SELECT count(*) AS rpm FROM Transaction TIMESERIES 1 minute)`
Compare to a computed value	`… WHERE duration > (SELECT average(duration) FROM Transaction)`
Combine two data types	`FROM Transaction JOIN (FROM ProcessSample SELECT … FACET containerId) ON containerId SELECT …`
Pull a value out of text	`WITH aparse(message, 'took * ms') AS ms` · `capture(message, r'.*took (?P<ms>\d+) ms.*')`
Reuse an expression	`FROM Transaction WITH round(duration * 1000) AS ms SELECT average(ms)`

I want to…	Write
By hour of day / weekday	`… FACET hourOf(timestamp)` · `FACET weekdayOf(timestamp)`
Local time zone	`… WITH TIMEZONE 'Asia/Kolkata'`
Raw rows to export	`SELECT timestamp, name, duration FROM Transaction LIMIT MAX`
What data exists	`SHOW EVENT TYPES` · `SELECT keyset() FROM Transaction`
Several types at once	`FROM Transaction, TransactionError SELECT count(*) FACET eventType()`
Run from a script	NerdGraph `nrql(query: "…") { results }` · `newrelic nrql query --accountId … --query "…"`

Sources

- New Relic docs — NRQL syntax, clauses and functions
- New Relic docs — Introduction to NRQL
- New Relic docs — NRQL math (SELECT)
- New Relic docs — Subqueries in NRQL, Subquery joins and Nested aggregation
- New Relic docs — Rate limits for NRQL queries and Query limits
- New Relic docs — NerdGraph NRQL tutorial and Async queries
- New Relic docs — Create NRQL alert conditions
- New Relic — What's new: NRQL string functions

NRQL and the New Relic UI change often. Syntax in this guide was checked against the documentation above at the time of writing; screens marked "Expected view" are redrawn for training.